

BILL OF MATERIALS

Component	Description	Quantity
Stepper motor (M1)	NEMA 17, 1.8° step angle	1
Motor driver (DR1)	A4988 or DRV8825 stepper driver	1
Microcontroller (MOD1)	Arduino Uno	1
Encoder (MOD2)	Rotary encoder for position feedback	1
Temperature sensor (S1)	LM35 temperature sensor for overheating detection	1
Current sensor (S2)	ACS712 current sensor for current monitoring	1

```

1  #include <PID_v1.h>
2
3  // PID Variables
4  double Setpoint, Input, Output;
5  double Kp = 2.0, Ki = 0.5, Kd = 1.0;
6  PID myPID(&Input, &Output, &Setpoint, Kp, Ki, Kd, DIRECT);
7
8  // Encoder Feedback
9  volatile long encoderPos = 0;
10 void updateEncoder() {
11     encoderPos++;
12 }
13
14 void setup() {
15     // Initialize PID
16     Setpoint = 1000; // Desired position
17     myPID.SetMode(AUTOMATIC);
18
19     // Attach Encoder Interrupt
20     attachInterrupt(digitalPinToInterrupt(2), updateEncoder, RISING);
21
22     // Initialize Serial for Monitoring
23     Serial.begin(9600);
24 }
25
26 void loop() {
27     // Read Encoder Position
28     Input = encoderPos;
29
30     // Compute PID
31     myPID.Compute();
32
33     // Drive Stepper Motor
34     analogWrite(9, Output);
35
36     // Monitor Temperature and Current
37     float temperature = analogRead(A0) * 0.488; // LM35
38     float current = (analogRead(A1) - 512) * 0.185; // ACS712
39
40     // Predictive Failure Logic
41     if (temperature > 60.0 || current > 1.5) {
42         Serial.println("Warning: Motor failure predicted!");
43     }
44 }

```

Fig. 3: Arduino code snippet

Bill of Materials table. The prototype is shown in Fig. 1.

PID control algorithm

Once the PID formula was developed, the design, proceeded with material selection. The PID controller is a feedback loop mechanism widely used in industrial control

systems. It calculates an error value as the difference between a desired setpoint (SP) and a measured process variable (PV). The controller then attempts to minimise this error by adjusting the control inputs accordingly. For further details and the PID formula, refer to standard textbooks or verified online resources.

Tuning the PID parameters is crucial for effective control. The proportional gain (Kp) helps reduce steady-state error but may cause oscillations if set too high. The integral gain (Ki) eliminates residual error by

accumulating past errors over time. The derivative gain (Kd) improves system stability and reduces overshoot by responding to the rate of change in error.

Advanced PID techniques improve performance in complex environments. Anti-windup mechanisms prevent excessive

accumulation of the integral term when the actuator reaches its limits. Cascade control uses multiple PID loops in a hierarchical structure to manage multi-stage systems more effectively. Feedforward control anticipates known disturbances and compensates for them in advance.

Circuit and working

Fig. 2 shows the circuit diagram of the predictive failure control system for stepper motors. It is built around an Arduino Uno board (MOD1), an encoder (MOD2), a temperature sensor (S1), a current sensor (S2), a motor driver (DR1), and several additional components.

The A4988 stepper motor driver connects to the Arduino's digital pins to handle STEP, DIR, and ENABLE signals. The rotary encoder is connected to interrupt pins D2 and D3 for precise position feedback. The LM35 temperature sensor provides an analogue signal to pin A0, while the ACS712 current sensor is connected to pin A1.

Simulation in Proteus

Proteus is a powerful simulation platform for microcontroller-based systems. To simulate the PID-controlled stepper motor system, configure all components in Proteus—including the Arduino Uno, A4988 driver, stepper motor, rotary encoder, LM35 sensor, and ACS712 current sensor—based on the circuit diagram.

Load the Arduino Uno with the HEX file generated in the Arduino IDE to execute the programmed logic. The simulation results demonstrate that the motor's position closely follows the desired setpoint with minimal error, confirming the effectiveness of the PID control. Failure conditions such as overheating and overcurrent are also